

Basic Level Questions

1. What is an exception in Java?

Answer:

An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions.

- **Example:** Division by zero, invalid array index.

```
int a = 10, b = 0;
```

```
System.out.println(a / b); // Throws ArithmeticException
```

2. What is the difference between checked and unchecked exceptions?

Feature	Checked Exception	Unchecked Exception
Compile-time check	Yes	No
Subclass of	Exception (except RuntimeException)	RuntimeException
Example	IOException, SQLException	NullPointerException, ArithmeticException

3. What are try, catch, and finally blocks?

Answer:

- **try:** Code that might throw an exception.
- **catch:** Code that handles the exception.
- **finally:** Code that always executes (cleanup code).

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
} finally {  
    System.out.println("This block always executes");  
}
```

4. What is the difference between throw and throws?

Answer:

- **throw:** Used to explicitly throw an exception.
- **throws:** Declares exceptions that a method might throw.

```
void divide(int a, int b) throws ArithmeticException {  
    if(b == 0) throw new ArithmeticException("Divide by zero");  
}
```

5. What is the difference between Error and Exception?

Answer:

- **Error:** Serious problems outside the application's control (e.g., OutOfMemoryError).
- **Exception:** Problems that can be handled in the program (e.g., IOException).

Intermediate Level Questions

6. Can we have multiple catch blocks?

Answer:

Yes, a try block can have multiple catch blocks to handle different types of exceptions.

```

try {
    int[] arr = new int[5];
    arr[10] = 50;
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Array index error");
} catch (Exception e) {
    System.out.println("Generic exception");
}

```

7. What is a finally block? Can it be omitted?

Answer:

- **finally:** Always executes after try-catch, used for cleanup.
- **Omitted:** Yes, but not recommended if resources need closing.

```

FileReader fr = null;
try {
    fr = new FileReader("test.txt");
} catch (IOException e) {
    e.printStackTrace();
} finally {
    if(fr != null) fr.close();
}

```

8. What is the difference between throw and throws? (*Intermediate nuance*)

Answer:

- **throw:** Can throw a single exception at a time.
- **throws:** Can declare multiple exceptions separated by commas.

```
void readFile() throws IOException, SQLException { }
```

9. Can we have try without catch?

Answer:

Yes, if a finally block is present.

```
try {  
    int a = 10 / 0;  
} finally {  
    System.out.println("Finally executes");  
}
```

10. What is the difference between Exception and RuntimeException?

Answer:

- **Exception:** Checked exceptions must be handled or declared.
- **RuntimeException:** Unchecked exceptions, optional to handle.

Advanced Level Questions

11. What is the difference between getMessage(), toString(), and printStackTrace() in Exception class?

Answer:

- getMessage(): Returns the error message.
- toString(): Returns the exception name + message.
- printStackTrace(): Prints the stack trace in console.

```
try {  
    int a = 10/0;  
} catch(ArithmeticException e) {  
    System.out.println(e.getMessage());  
}
```

```
System.out.println(e.toString());  
e.printStackTrace();  
}
```

12. How to create a custom exception?

```
class MyException extends Exception {  
    MyException(String message) { super(message); }  
}
```

```
public class Test {  
    static void validate(int age) throws MyException {  
        if(age < 18) throw new MyException("Age must be >= 18");  
    }  
    public static void main(String[] args) {  
        try { validate(15); }  
        catch(MyException e) { System.out.println(e.getMessage()); }  
    }  
}
```

13. Explain exception propagation.

Answer:

If an exception is not caught in the current method, it is propagated to the caller method, and continues until main method. If uncaught, program terminates.

14. Can we catch multiple exceptions in a single catch block?

Answer:

Yes, using multi-catch (Java 7+).

```
try {
```

```

int a = 10/0;

String s = null;

s.length();
} catch(ArithmeticException | NullPointerException e) {

    System.out.println("Exception handled: " + e);

}

```

15. What are suppressed exceptions?

Answer:

Exceptions thrown in a try-with-resources block when closing resources are called **suppressed exceptions**. Can be accessed using `getSuppressed()`.

```

try(FileReader fr = new FileReader("file.txt")) {

    throw new Exception("Main exception");

} catch(Exception e) {

    for(Throwable t : e.getSuppressed()) {

        t.printStackTrace();

    }

}

```

16. Difference between checked, unchecked, and errors in terms of handling.

Type	Must Handle?	When Thrown?	Examples
Checked Exception	Yes	Compile-time	IOException, SQLException
Unchecked Exception	Optional	Runtime	NullPointerException, ArrayIndexOutOfBoundsException
Error	Cannot handle	Runtime	OutOfMemoryError, StackOverflowError

17. What is try-with-resources?

Answer:

- Automatically closes resources that implement AutoCloseable.

```
try (BufferedReader br = new BufferedReader(new FileReader("test.txt"))) {  
    System.out.println(br.readLine());  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

18. Difference between exception chaining and suppressed exceptions?

- **Exception chaining:** Wrapping one exception inside another using Throwable initCause().
- **Suppressed exceptions:** Exceptions thrown while closing resources in try-with-resources.

19. Can we override a method that throws exceptions?

Answer:

- Overridden method **cannot throw broader checked exceptions** than the superclass.
- Can throw unchecked exceptions.

```
class Parent {  
    void show() throws IOException {}  
}  
  
class Child extends Parent {  
    void show() {} // allowed  
    void show2() throws Exception {} // Not allowed  
}
```

20. How to log exceptions properly in real-world applications?

- Use logging frameworks: **Log4j**, **SLF4J**, **java.util.logging**
- Avoid using `printStackTrace()` in production.
- Include context info: timestamp, method, parameters, exception message.